

UNIVERSIDAD DEL VALLE DE GUATEMALA
Facultad de Ingeniería

Publicación de un paquete de Python en TestPyPI

Diego Linares

Andy Fuentes

Christian Echeverria

Diederich Solis

Machine Learning Engineering

Septiembre 1 de 2026

Paquete publicado:
<https://test.pypi.org/project/act3-pipeline-mlops/>

Qué decidimos publicar

Lo primero que tuvimos que aclarar fue *qué* se sube a PyPI, porque nuestra primera idea fue publicar el trabajo de la Actividad 4, que está dockerizado. No se puede, y por una razón de fondo: PyPI distribuye **paquetes de Python**; las imágenes de contenedor van a un registro de imágenes como Docker Hub. Son dos cosas distintas.

Lo que sí es publicable de la Actividad 4 es la **librería que vive adentro de sus imágenes**: el paquete `act3_pipeline`, que armamos en la Actividad 1 y que contiene el pipeline completo con la calibración de hiperparámetros de la Actividad 3.

Y ahí encontramos la mejor justificación para hacerlo. Hoy los dos `Dockerfile` de la Actividad 4 instalan la librería copiando un archivo a mano dentro del repositorio:

```
COPY libreria/act3_pipeline-0.1.0-py3-none-any.whl /tmp/
RUN pip install --no-cache-dir /tmp/act3_pipeline-0.1.0-py3-none-any.whl
```

Con el paquete publicado, eso se vuelve una dependencia normal, igual que cualquier otra:

```
RUN pip install act3-pipeline-mlops
```

Se acabó el `.whl` versionado a mano dentro del repositorio.

La estructura del proyecto

Seguimos la estructura del tutorial oficial (Python Packaging Authority, s.f.-a): disposición `src/` y toda la configuración en un solo `pyproject.toml`, sin `setup.py` ni `setup.cfg`.

```
Tarea4/
|-- pyproject.toml          # toda la configuracion del paquete
|-- README.md              # se muestra como descripcion en TestPyPI
|-- LICENSE                # MIT
|-- .env                   # el token (NO se sube al repositorio)
|-- .gitignore
|-- src/
|   '-- act3_pipeline/      # el paquete
|       |-- __init__.py
|       |-- data.py         transformers.py    pipeline.py
|       |-- tuning.py       evaluation.py     cli.py
|       '-- datasets/champions_league_matches.csv
'-- dist/                  # lo que genera la construccion
```

El `pyproject.toml`

```
[build-system]
requires = ["hatchling >= 1.26"]
build-backend = "hatchling.build"
```

```

[project]
name = "act3-pipeline-mlops"
version = "0.1.0"
authors = [
    { name = "Diego Linares", email = "lin221256@uvg.edu.gt" },
    { name = "Andy Fuentes" },
    { name = "Christian Echeverria" },
    { name = "Diederich Solis" },
]
description = "Pipeline de scikit-learn con calibracion de hiperparametros ..."
readme = "README.md"
requires-python = ">=3.10"
license = "MIT"
license-files = ["LICENSE"]
classifiers = [
    "Programming Language :: Python :: 3",
    "Operating System :: OS Independent",
    "Intended Audience :: Education",
    "Topic :: Scientific/Engineering :: Artificial Intelligence",
]
dependencies = [
    "scikit-learn>=1.3", "pandas>=2.0", "numpy>=1.24", "scipy>=1.10",
]

[project.scripts]
act3-demo = "act3_pipeline.cli:main"

# El nombre de la distribucion no coincide con el del paquete,
# asi que hay que decirle a hatchling donde buscarlo.
[tool.hatch.build.targets.wheel]
packages = ["src/act3_pipeline"]

```

Tres detalles que nos costaron y vale la pena anotar:

- **El nombre de la distribución no es el nombre del paquete.** Se instala como `act3-pipeline-mlops` pero se importa como `act3_pipeline`. Al no coincidir, hubo que declarar `[tool.hatch.build.targets.wheel]` para que el constructor encontrara el código.
- **El nombre tiene que ser único en todo el índice.** Le agregamos el sufijo `-mlops` porque `act3-pipeline` a secas se podía chocar con otro proyecto.
- **`dependencies` no es lo mismo que `requirements.txt`.** Aquí van rangos abiertos (`>=`) porque quien instale el paquete puede tener otras librerías que también necesitan resolverse; las versiones exactas se fijan en la aplicación, no en la librería.

Construcción

```
python -m build
```

Genera los dos formatos que espera PyPI:

Archivo	Tamaño	Qué es
act3_pipeline_mlops-0.1.0-py3-none-any.whl	24 KB	Distribución construida; es la que <code>pip</code> instala directo
act3_pipeline_mlops-0.1.0.tar.gz	20 KB	Código fuente, por si hay que re-construir

Verificamos que el dataset viajara adentro, porque de nada sirve el paquete si hay que descargar los datos aparte:

```
act3_pipeline/__init__.py
act3_pipeline/cli.py      data.py      evaluation.py
act3_pipeline/pipeline.py tuning.py   transformers.py
act3_pipeline/datasets/champions_league_matches.csv  <-- los datos van adentro
act3_pipeline_mlops-0.1.0.dist-info/METADATA
act3_pipeline_mlops-0.1.0.dist-info/entry_points.txt <-- el comando act3-demo
act3_pipeline_mlops-0.1.0.dist-info/licenses/LICENSE
```

Antes de subir nada, validamos la metadata:

```
python -m twine check dist/*
```

```
Checking dist\act3_pipeline_mlops-0.1.0-py3-none-any.whl: PASSED
```

```
Checking dist\act3_pipeline_mlops-0.1.0.tar.gz: PASSED
```

Publicación en TestPyPI

Siguiendo la guía de TestPyPI (Python Packaging Authority, s.f.-b), creamos la cuenta, generamos un token de API y subimos con `twine`.

El manejo del token

Un token de PyPI permite publicar en nombre de uno, así que no puede quedar escrito en un comando ni en el historial de la terminal. Lo guardamos en un archivo `.env` excluido por `.gitignore`:

```
TWINE_USERNAME=__token__
TWINE_PASSWORD=pypi-AgENdGVzdC5weXBpLm9yZw...
```

El usuario es literalmente `__token__`, no el correo. Un script de PowerShell carga esas variables, construye, valida y sube, de modo que el token nunca aparece en pantalla:

```
.\subir_a_testpypi.ps1
```

```
Token cargado desde .env (usuario: __token__)
[1/3] Construyendo el paquete...
[2/3] Validando la metadata...
[3/3] Subiendo a TestPyPI...
```

Verificación: instalarlo desde el índice

Publicar no prueba nada; había que instalarlo desde TestPyPI en limpio. Aquí está el error más común de esta guía, y lo cometimos:

```
pip install --index-url https://test.pypi.org/simple/ act3-pipeline-mlops
```

Eso falla. TestPyPI es un índice *separado* y no tiene scikit-learn ni pandas, así que pip no puede resolver las dependencias. Hay que decirle que las busque en el PyPI real:

```
pip install --index-url https://test.pypi.org/simple/ \
            --extra-index-url https://pypi.org/simple/ \
            act3-pipeline-mlops
```

```
Downloading https://test-files.pythonhosted.org/packages/.../
act3_pipeline_mlops-0.1.0-py3-none-any.whl (24 kB)
Successfully installed act3-pipeline-mlops-0.1.0
```

Y comprobamos que el paquete instalado desde el índice sirve de verdad:

```
>>> import act3_pipeline
distribucion instalada: 0.1.0
se importa como       : act3_pipeline 0.1.0
dataset incluido      : 115 entrenamiento / 29 prueba
pipeline              : ['preprocesamiento', 'seleccion', 'modelo']
```

El comando de consola también quedó instalado y corre el pipeline completo:

```
act3-demo --busqueda aleatoria
```

```
RESULTADO - accuracy=0.690 f1_macro=0.631
```

```
[OK] Pipeline calibrado y evaluado correctamente en este equipo.
```

Reflexión: cómo empaquetar un proyecto de Machine Learning

La pregunta de la tarea es qué conviene más entre Docker, *pickles* y paquetes. Después de haber usado los tres en el curso, nuestra conclusión es que **la pregunta está mal planteada**: no compiten entre sí porque no empaquetan lo mismo.

Herramienta	Qué empaqueta	Dónde la usamos
Paquete (<i>wheel</i>)	El código y sus dependencias declaradas	Actividad 1, y hoy en TestPyPI
Pickle / joblib	El modelo entrenado : los coeficientes aprendidos	El modelo.joblib de la Actividad 4
Docker	El ambiente completo : sistema operativo, intérprete, librerías y código	Las tres imágenes de la Actividad 4

Los tres, en capas

Lo que nos funcionó es usarlos apilados, cada uno en su papel:

1. El **código** se publica como paquete, con sus dependencias declaradas y una versión.
2. El paquete se **instala dentro de la imagen**, que además fija la versión del intérprete y del sistema operativo.
3. El **modelo entrenado vive fuera de la imagen**, en un volumen o en un registro de modelos, porque es estado y no código.

Ese tercer punto lo aprendimos por decisión propia en la Actividad 4: nos pareció mala idea hornear el modelo dentro de la imagen porque reentrenar habría obligado a reconstruir y redistribuir la imagen completa.

Por qué el pickle es el eslabón peligroso

De las tres piezas, la que más problemas nos dio fue el modelo serializado, y por una razón que solo se entiende cuando pasa: **un pickle no guarda el código de sus clases, guarda la ruta para importarlas**. Nuestro pipeline contiene dos transformadores propios, `PorcentajeATexto` y `RatioATexto`. Cuando la API carga el `modelo.joblib`, Python busca esas clases en el módulo `act3_pipeline`; si no las encuentra, el archivo es inservible.

Por eso las dos imágenes de la Actividad 4 instalan el mismo paquete aunque la API no entrene nada, y por eso ambas fijan las mismas versiones exactas. Un modelo serializado sin el paquete que lo acompaña es una bomba de tiempo: funciona hoy en la máquina de quien lo entrenó y falla mañana en cualquier otra.

Se suma que *pickle* ejecuta código al deserializar, así que cargar uno de origen desconocido es un riesgo de seguridad, y que está atado a la versión de la librería con la que se creó.

Nuestra recomendación

- **Si el código se reutiliza entre proyectos** —un pipeline, transformadores, utilidades— va como **paquete**. Es lo que hicimos hoy, y elimina el `.whl` copiado a mano.
- **Si hay que reproducir un experimento o servir un modelo**, va en **Docker**. Es lo único que garantiza el mismo resultado en Windows y en macOS, algo que sufrimos en la Actividad 3 antes de contenerizar.
- **El modelo entrenado** no va en el repositorio ni horneado en la imagen: va a un volumen o, mejor, a un **registro de modelos** con su versión y sus métricas. Al llevar el pipeline a Databricks vimos cómo MLflow hace exactamente eso, y es bastante más ordenado que nuestro volumen con un `metadatos.json` al lado.

Si tuviéramos que quedarnos con una sola idea: **empaquetar el código es fácil y ya está resuelto**; lo difícil de un proyecto de Machine Learning es empaquetar el modelo y los datos, que son las dos piezas que cambian solas y que ninguna de estas tres herramientas versiona bien por sí sola.

Referencias

- Python Packaging Authority. (s.f.-a). *Packaging Python projects*. Python Packaging User Guide. <https://packaging.python.org/en/latest/tutorials/packaging-projects/>
- Python Packaging Authority. (s.f.-b). *Using TestPyPI*. Python Packaging User Guide. <https://packaging.python.org/en/latest/guides/using-testpypi/>
- Python Packaging Authority. (s.f.-c). *Writing your pyproject.toml*. Python Packaging User Guide. <https://packaging.python.org/en/latest/guides/writing-pyproject-toml/>
- Python Software Foundation. (s.f.). *pickle — Python object serialization*. Python documentation. <https://docs.python.org/3/library/pickle.html>
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems*, 28.